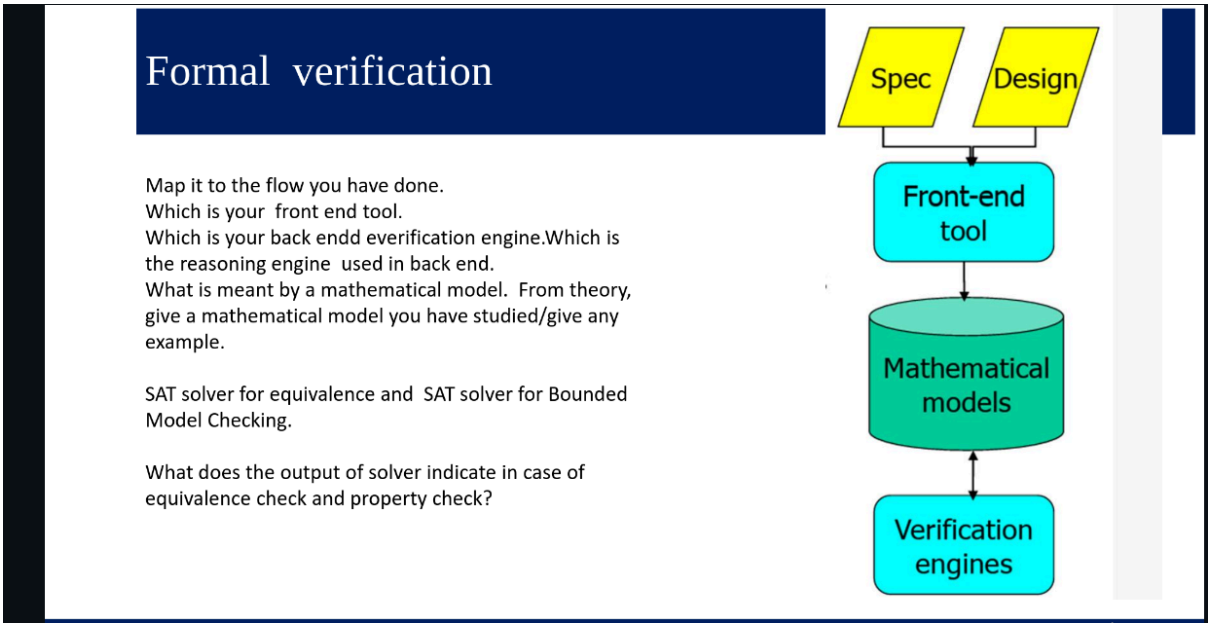
Question 0)



Front-end tool: y**osys**. Its job is to parse the Verilog RTL, elaborate the design hierarchy, and synthesize it into an internal mathematical representation (like an Abstract Syntax Tree (AST Representation) or a mapped netlist).
Back-end verification engine & Reasoning engine: ABC or the built-in Yosys `sat` command. The reasoning engine running inside them is a **SAT Solver** (like MiniSAT or Z3).
Mathematical Model: A mathematical model is an abstract representation of the hardware's behavior over time using logic formulas and state variables.

- *Example to give:* A **Kripke Structure** or a Finite State Machine (FSM). It is defined as a tuple (S,S0,R,L), where S is the set of all possible states, S0 is the initial state, R is the transition relation (next-state logic), and L tracks the outputs.
- Other example from class is BDD or BMC

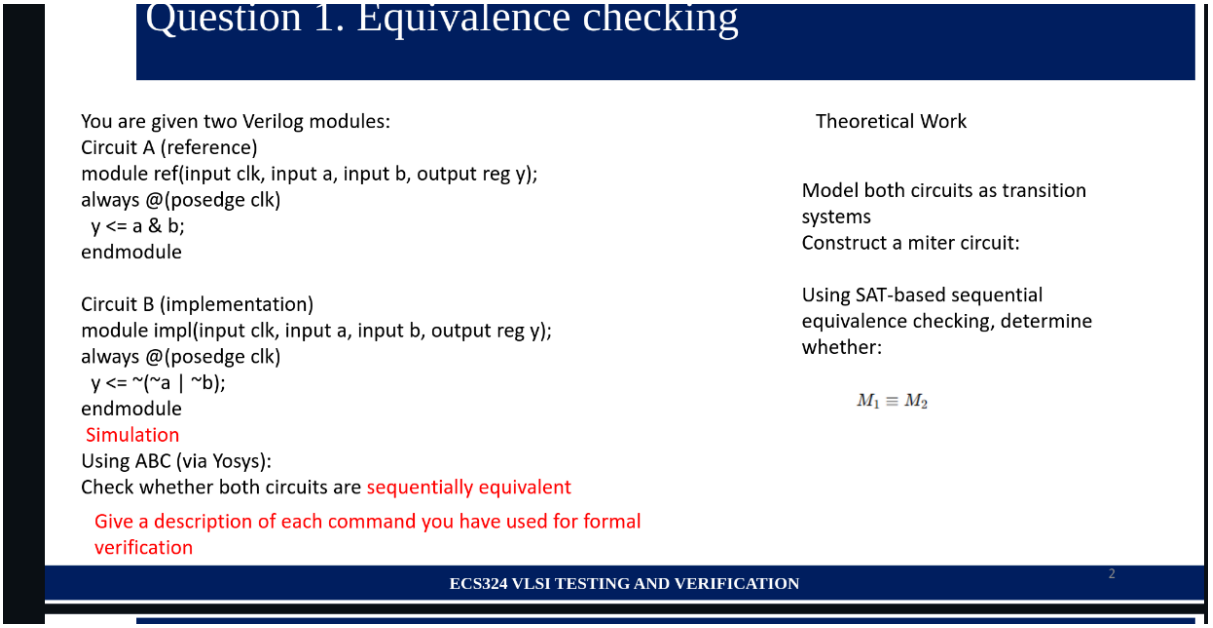
SAT for Equivalence vs. BMC:

- *Equivalence:* The SAT solver checks if a "Miter" circuit (two circuits XORed together) can ever output a 1.
- *BMC (Bounded Model Checking):* The SAT solver unrolls the design for k clock cycles and checks if a specific property violation can occur within those k steps.

Output of the Solver:

- **SAT (Satisfiable):** The solver found a valid path. In equivalence checking, this means a difference was found (they are NOT equivalent). In property checking, it means a counter-example was found (a bug exists).
- **UNSAT (Unsatisfiable):** The solver proved it is impossible. In equivalence, it means the circuits ARE equivalent. In property checking, it means the property holds true (no bug found).

Question 1)



read_verilog circuit_1.v

read_verilog circuit_2.v

proc

opt

equiv_make ref imp equiv_miter

hierarchy -top equiv_miter

flatten

equiv_simple

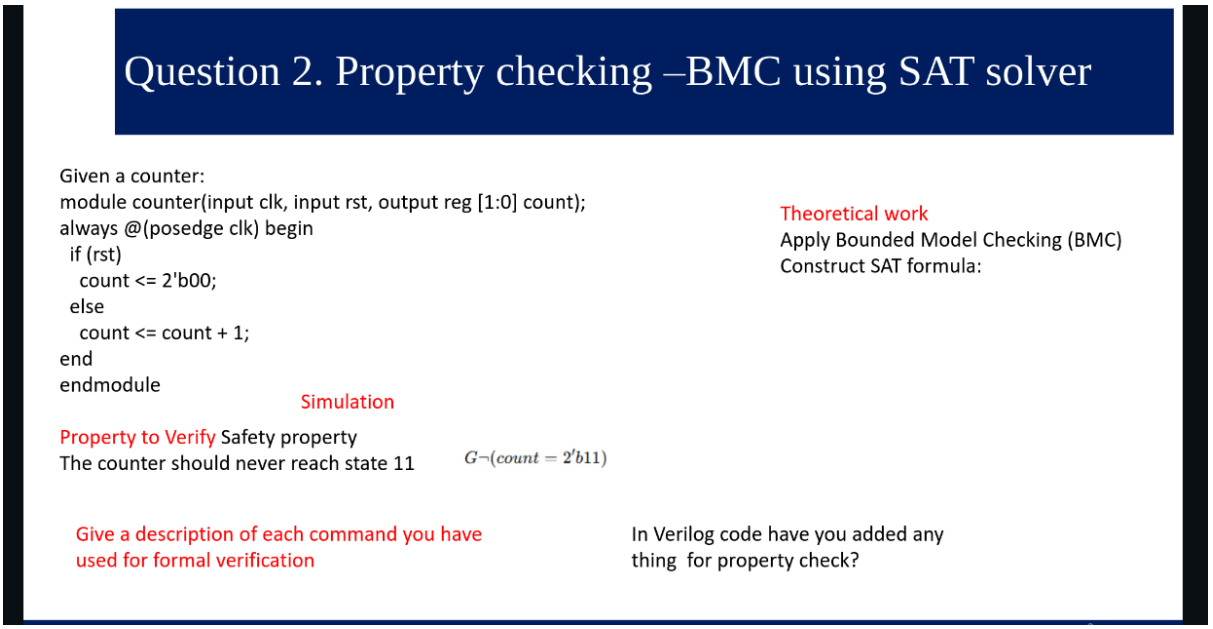
equiv_induct

equiv_status -assert

Output :

```
Removing unused module `\ref'.
Removed 2 unused modules.

7. Executing FLATTEN pass (flatten design).

8. Executing EQUIV_SIMPLE pass.
Found 1 unproven $equiv cells (1 groups) in equiv_miter:
  Trying to prove $equiv for \y: failed.
Proved 0 previously unproven $equiv cells.

9. Executing EQUIV_INDUCT pass.
Found 1 unproven $equiv cells in module equiv_miter:
  Proving existence of base case for step 1. (78 clauses over 35 variables)
  Proving induction step 1. (173 clauses over 74 variables)
  Proof for induction step holds. Entire workset of 1 cells proven!
Proved 1 previously unproven $equiv cells.

10. Executing EQUIV_STATUS pass.
Found 1 $equiv cells in equiv_miter:
  Of those cells 1 are proven and 0 are unproven.
  Equivalence successfully proven!

End of script. Logfile hash: 686824a361, CPU: user 0.02s system 0.01s, MEM: 9.20 MB peak
Yosys 0.33 (git sha1 2584903a060)
Time spent: 30% 4x opt_expr (0 sec), 19% 3x opt_merge (0 sec), ...
```

Question 2)



read_verilog -sv -D FORMAL countersques.v

prep -top counter

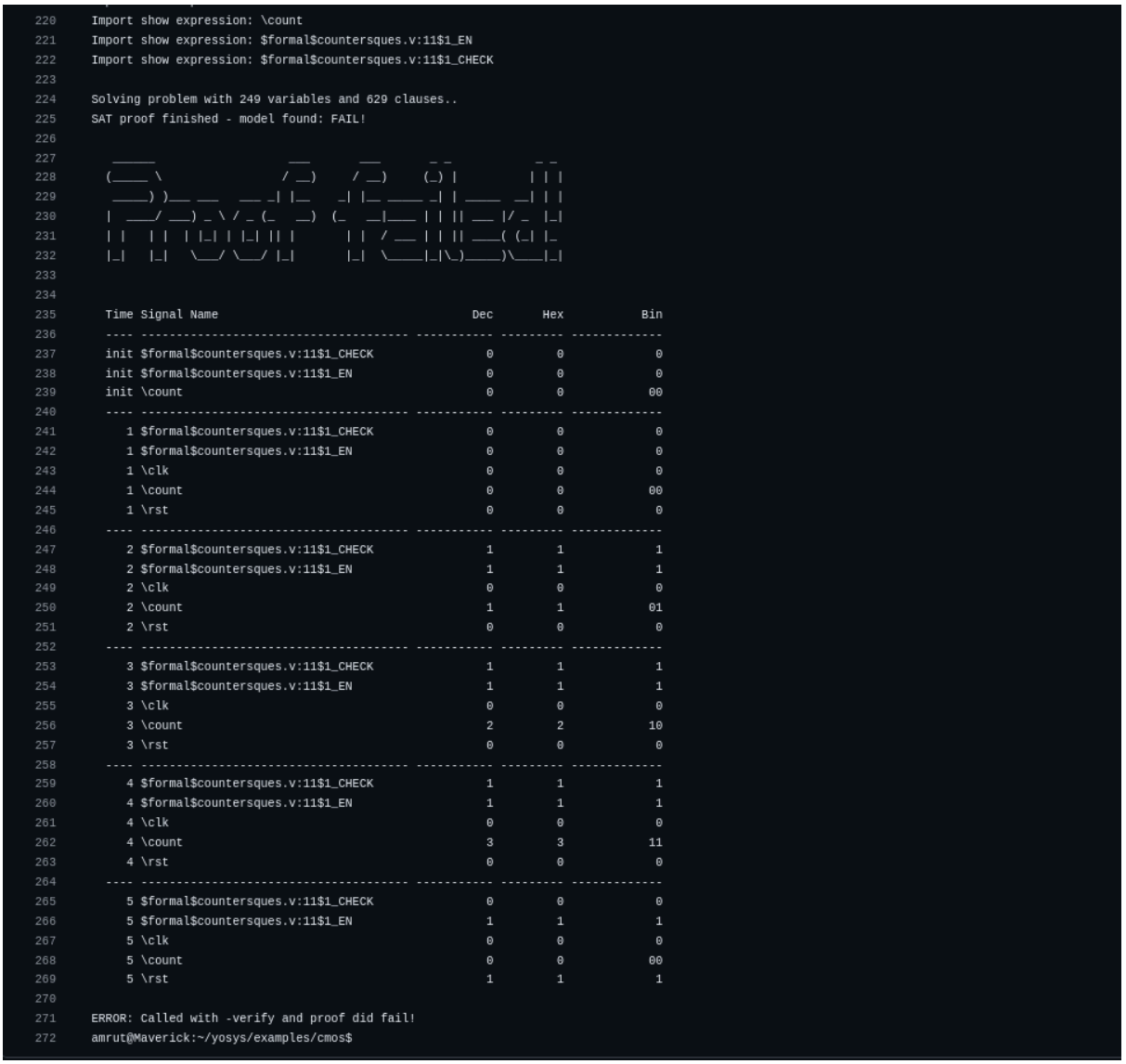
sat -verify -prove-asserts -seq 5 -show-inputs -show-outputs -show-regs

RTL :

```
module counter(input clk, input rst, output reg [1:0] count);
    always @(posedge clk) begin
        if (rst)
            count <= 2'b00;
        else
            count <= count + 1;
    end

   `ifdef FORMAL
    always @(posedge clk) begin
        // The property: The counter should never reach state 11
```

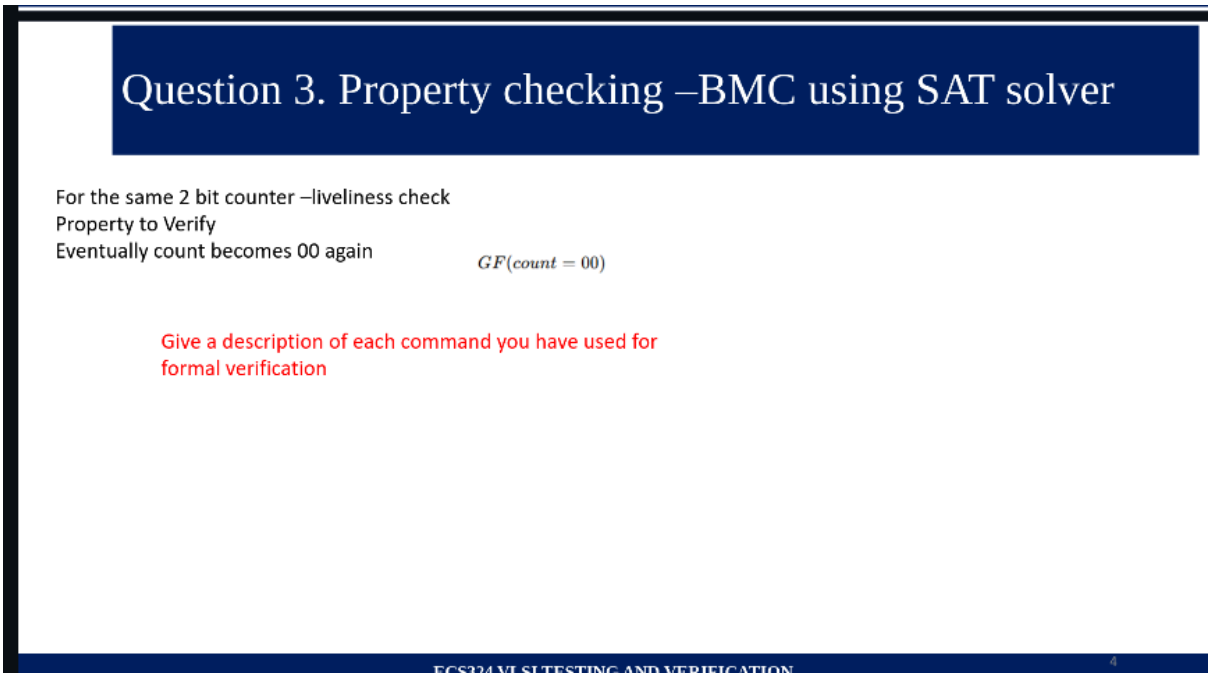
```
        assert(count != 2'b11);
    end
    `endif
endmodule
```

Output :



```
Import show expression: \count
Import show expression: $formal$countersques.v:11$1_EN
Import show expression: $formal$countersques.v:11$1_CHECK

Solving problem with 249 variables and 629 clauses..
SAT proof finished - model found: FAIL!

  _____                    ___       ___       _ _           _ _
 (_____ \                 / __)     / __)     (_) |         | | |
  _____) )___ ___   ___ _| |__    _| |__ _____ _| | _____  __| | |
 |  ____/ ___) _ \ / _ (_   __)  (_   __|____ | | || ___ |/ _  |_|
 | |   | |  | |_| | |_| || |       | |  / ___ | | || ____( (_| |_
 |_|   |_|   \___/ \___/ |_|       |_|  \_____|_|\_)_____)\____|_|


  Time Signal Name                                 Dec       Hex           Bin
  ---- ----------------------------------- ---------- --------- -------------
  init $formal$countersques.v:11$1_CHECK             0         0             0
  init $formal$countersques.v:11$1_EN                0         0             0
  init \count                                        0         0            00
  ---- ----------------------------------- ---------- --------- -------------
     1 $formal$countersques.v:11$1_CHECK             0         0             0
     1 $formal$countersques.v:11$1_EN                0         0             0
     1 \clk                                          0         0             0
     1 \count                                        0         0            00
     1 \rst                                          0         0             0
  ---- ----------------------------------- ---------- --------- -------------
     2 $formal$countersques.v:11$1_CHECK             1         1             1
     2 $formal$countersques.v:11$1_EN                1         1             1
     2 \clk                                          0         0             0
     2 \count                                        1         1            01
     2 \rst                                          0         0             0
  ---- ----------------------------------- ---------- --------- -------------
     3 $formal$countersques.v:11$1_CHECK             1         1             1
     3 $formal$countersques.v:11$1_EN                1         1             1
     3 \clk                                          0         0             0
     3 \count                                        2         2            10
     3 \rst                                          0         0             0
  ---- ----------------------------------- ---------- --------- -------------
     4 $formal$countersques.v:11$1_CHECK             1         1             1
     4 $formal$countersques.v:11$1_EN                1         1             1
     4 \clk                                          0         0             0
     4 \count                                        3         3            11
     4 \rst                                          0         0             0
  ---- ----------------------------------- ---------- --------- -------------
     5 $formal$countersques.v:11$1_CHECK             0         0             0
     5 $formal$countersques.v:11$1_EN                1         1             1
     5 \clk                                          0         0             0
     5 \count                                        0         0            00
     5 \rst                                          1         1             1

ERROR: Called with -verify and proof did fail!
amrut@Maverick:~/yosys/examples/cmos$
```

Question 3)



RTL :

```
module counter(input clk, input rst, output reg [1:0] count);
   initial count = 2'b00;

   always @(posedge clk) begin
      if (rst)
         count <= 2'b00;
      else
         count <= count + 1;
   end

   `ifdef FORMAL
   reg [2:0] cycles_since_zero;
   initial cycles_since_zero = 0;

   always @(posedge clk) begin
```

```
        if (rst || count == 2'b00) begin
            cycles_since_zero <= 0;
        end else begin
            cycles_since_zero <= cycles_since_zero + 1;
        end

        assert(cycles_since_zero < 4);
    end
    `endif
endmodule

Yosys;

read_verilog -sv -D FORMAL counter_live.v
prep -top counter
sat -verify -prove-asserts -seq 8 -show-regs
```

Output :

```
Import show expression: $formal$counter_live.v:24$1_CHECK
Import show expression: \cycles_since_zero

Solving problem with 929 variables and 2453 clauses..
SAT proof finished - no model found: SUCCESS!

                   /$$$$$$      /$$$$$$$$     /$$$$$$$
                  /$$__  $$    | $$_____/    | $$__  $$
                 | $$  \ $$    | $$          | $$  \ $$
                 | $$  | $$    | $$$$$       | $$  | $$
                 | $$  | $$    | $$__/       | $$  | $$
                 | $$/$$ $$    | $$          | $$  | $$
                 |  $$$$$$/ /$$| $$$$$$$$ /$$| $$$$$$$//$$
                  \____ $$$|__/|________/|__/|_______/|__/
                       \__/

End of script. Logfile hash: af0809eaf1, CPU: user 0.03s system 0.01s, MEM: 10.41 MB peak
Yosys 0.33 (git sha1 2584903a060)
Time spent: 31% 1x sat (0 sec), 20% 5x opt_expr (0 sec), ...
amrut@Maverick:~/yosys/examples/cmos$
```

Question 4)

Question 4. Optimization and equivalence check

To perform logic optimization on a given RTL design and verify that the optimized netlist is functionally equivalent to the original design using formal equivalence checking.

```
module top(input A, B, C, output F);
assign F = (A & B) | (A & B & C);
endmodule
```

Part A: Logic Optimization
Write a Yosys script to:
Read the RTL file
Prepare the design
Perform logic optimization
Generate the optimized netlist

Give a description of each command you have used for formal verification

Part B: Equivalence Checking
Write another Yosys script to:
Load the original RTL
Load the optimized netlist
Create a miter circuit
Perform equivalence checking

ECS324 VLSI TESTING AND VERIFICATION

Yosys to make netlist :

```
read_verilog ques4.v

prep -top top

opt

write_verilog top_opt.v
```

Yosys to do equiv check : (very similar to question 1)

read_verilog ques4.v

rename top top_ref

read_verilog top_opt.v

rename top top_impl

equiv_make top_ref top_impl equiv_miter

hierarchy -top equiv_miter

equiv_simple

equiv_status -assert

Output :

```
308
309     4. Executing HIERARCHY pass (managing design hierarchy).
310
311     4.1. Analyzing design hierarchy..
312     Top module:  \equiv_miter
313
314     4.2. Analyzing design hierarchy..
315     Top module:  \equiv_miter
316     Removing unused module `\top_impl'.
317     Removing unused module `\top_ref'.
318     Removed 2 unused modules.
319
320     5. Executing EQUIV_SIMPLE pass.
321     Found 1 unproven $equiv cells (1 groups) in equiv_miter:
322       Trying to prove $equiv for \F: success!
323     Proved 1 previously unproven $equiv cells.
324
325     6. Executing EQUIV_STATUS pass.
326     Found 1 $equiv cells in equiv_miter:
327       Of those cells 1 are proven and 0 are unproven.
328       Equivalence successfully proven!
329
330     End of script. Logfile hash: f617e5e869, CPU: user 0.01s system 0.01s, MEM: 8.85 MB peak
331     Yosys 0.33 (git sha1 2584903a060)
332     Time spent: 35% 4x read_verilog (0 sec), 28% 1x equiv_make (0 sec), ...
333     amrut@Maverick:~/yosys/examples/cmos$
```